

FIELD FLOW

Product Strategy & MVP Planning Guide

A beginner-friendly plan for turning four connected student products into one coherent contractor platform.

Scope: planning, architecture, data, UX, safety, and team delivery.

HOW TO USE THIS GUIDE

Build the MVP column first. Treat anything marked Future as a deliberate next step, not unfinished MVP work.

1. The product in one view

FieldFlow is a simple software platform for small service contractors. Its job is to help a contractor like John manage daily work, understand business activity, and get support - while giving FieldFlow staff a separate view of account health.

DESIGN NORTH STAR

One person should feel like they are using one product, even though the team built four connected areas.

John: the core contractor persona

John runs a small plumbing business. He is busy, often away from a desk, and wants clear answers rather than complicated software. The interface should therefore prioritize today's jobs, fast scheduling, familiar words, and short insights.

John needs	FieldFlow response
Know what is happening today	A contractor dashboard with today's jobs and a prominent + Add Job action.
Keep work organized	Scheduling and client records that use the same shared data.
See whether business is improving	A few clear analytics cards and a trend chart, not a wall of charts.
Get unstuck quickly	A support chatbot that answers help questions and safely retrieves basic account facts.
Be treated as one account	Login and permissions that show only John's Plumbing data.

MVP rule

An MVP is the smallest version that proves a useful product flow. For FieldFlow, the proof is: **John logs in, creates or views work, sees the related analytics, gets help, and internal staff can see account health.**

2. Four products, one ecosystem

Product	MVP: build now	Future enhancement: later
A. Scheduling	Create, edit, cancel, and reschedule jobs; manage clients; weekly schedule.	Route optimization, staff assignments, reminders, calendar sync.
B. Internal Ops Dashboard	View contractor accounts, usage/activity, filters, and a transparent "At Risk" flag.	Predictive churn model, outreach workflow, richer support history.
C. Support Chatbot	10-20 help intents or approved-doc answers; safe plan/job-count lookup; links to relevant pages.	Broader LLM conversation, voice, controlled write actions.
D. Customer Analytics	Jobs this week vs. last week, clients, repeat clients, a simple trend/service chart.	Forecasting, custom reports, deeper segmentation.

The linchpins

The contractor dashboard is the **UX linchpin**: it summarizes Scheduling and Analytics, then routes John to the full views. The shared backend/database is the **technical linchpin**: it ensures every product sees the same jobs, clients, accounts, and activity.

Contractor dashboard vs. Internal Ops Dashboard

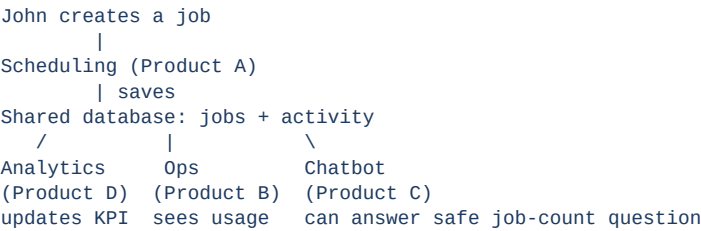
Contractor dashboard (John)	Internal Ops Dashboard (FieldFlow staff)
Today's schedule, upcoming work, quick "Add Job," small weekly KPIs, links to Schedule / Analytics / Help.	Account list, activity/usage signals, at-risk filtering, account details, cross-account view.
Question answered: "What do I need to do and how is my business doing?"	Question answered: "Which customers may need attention, and why?"
Only the logged-in contractor's account.	Authorized staff can view multiple accounts.

KEEP ROLE EXPERIENCES SEPARATE

Product B is not John's home screen. It is an internal tool with a different audience, data scope, and navigation.

3. Make the products behave as one

The best connections are not merely links. They pass useful context and rely on a shared source of truth.



High-value connection	How it works in the MVP
Scheduling -> Analytics	Saving a job changes the jobs-this-week count and trend the next time data is read/refreshed.
Scheduling -> Ops	A job/activity record contributes to whether the contractor appears active.
Chatbot -> Scheduling	"You have 4 jobs tomorrow" can include a link that opens tomorrow's schedule.
Analytics -> Scheduling	"Thursday is busiest" can link to Thursday preselected in the schedule.
Chatbot -> Account data	The bot retrieves John's plan or safe counts through authenticated backend queries - never by guessing.

Do not overbuild events

It is useful to think in events such as **JOB_CREATED** or **JOB_CANCELLED**, but the MVP does not need an enterprise event platform. A shared database plus clean queries is enough to demonstrate the same connected effect.

Login and roles

Role	Can do	MVP boundary
Contractor	View/manage own jobs and clients; view own analytics; use support chatbot.	Only records for their own account_id .
Internal staff	View authorized contractor accounts; inspect activity and at-risk signals.	No automatic impersonation; access only the staff functions required for the demo.

AUTHENTICATION VS. AUTHORIZATION

Authentication answers "Who are you?" Authorization answers "What data and actions may you access?" Both are needed.

4. Shared backend, live data, and architecture

Use one shared backend rather than separate databases for the four products. It provides a single version of John's data and makes the integrations believable.

```
React application(s)
|
Supabase Auth -> establishes user, account, role
|
Supabase API / application service
|
PostgreSQL source of truth
accounts | users | clients | jobs | usage_events
```

Core table	Why it exists
users	Links a signed-in person to a role and account.
accounts	The contractor business, plan, and account status.
clients	Customers belonging to a contractor account.
jobs	Scheduled work, service type, time, status, and client relationship.
usage_events	Activity used by Ops, such as login or job creation.

Live data: what it means

For the MVP, “live” can simply mean every product reads the current shared database instead of its own copied file. You may refresh after a save. Real-time subscriptions and instant push updates are a useful **future enhancement**, not a requirement for proving the concept.

Recommended stack

Layer	Recommendation	Reason
Frontend	React + TypeScript	Reusable screens/components; types catch shared-data mistakes.
Styling	CSS + shared design tokens	One familiar brand with room for product-specific layouts.
Data/auth	Supabase + PostgreSQL	Shared database, authentication, API-style access, and simple MVP setup.
Analytics	SQL + TypeScript; optional Python	Use SQL for shared data queries; use Python only when a real analysis/ML need exists.
Chatbot	TypeScript service/UI; optional LLM API	No Python requirement just because a chatbot uses AI.

JavaScript, TypeScript, and Python can interoperate

They do not need to read each other's source code. They agree on **JSON over an API**. TypeScript is the recommended default for a multi-person web app; JavaScript is a good simpler alternative; Python is appropriate for a specific data/ML task, not because it is “more advanced.”

5. Shared data: JSON, CSV, APIs, and contracts

Thing	Best use in FieldFlow	Not the same as
JSON	Shared mock/seed data and API request/response payloads. Handles related objects well.	An API or a production database.
CSV	Spreadsheet inspection, exports, pivot tables, or a simple flat data set.	Nested application data.
API	The doorway applications use to request or update data; usually sends JSON.	A data format.
PostgreSQL database	Long-term source of truth for shared, changing data.	A file that each product copies.

PRACTICAL PROGRESSION

Synthetic JSON -> seed/import -> PostgreSQL/Supabase -> API returns JSON -> optional CSV export for analysis.

The FieldFlow data contract

Write a short team standard before serious development. This prevents the real problem - different meanings and field names - even if team members use different languages.

Standard	Recommended rule
Terminology	Use Client , Job , Account , and Contractor consistently.
Field names	Use snake_case in shared JSON/API/database fields.
IDs	Use readable MVP IDs: ACC001 , CL001 , JOB001 , USR001 . Treat IDs as strings.
Dates/times	Store ISO dates: YYYY-MM-DD ; time as HH:MM ; display them in friendly local UI form.
Statuses	Job: Scheduled, Completed, Cancelled, Rescheduled. Account: Active, At Risk, Inactive.
Business definitions	Example: repeat client = more than one completed job. At risk = current 7-day activity at least 60% lower than the previous 7 days.
Responses	Use the same success/error JSON envelope in every API response.

```
{
  "job_id": "JOB001",
  "account_id": "ACC001",
  "client_id": "CL001",
  "service_type": "Leak Repair",
  "scheduled_date": "2026-08-20",
  "scheduled_time": "14:00",
  "status": "Scheduled"
}
```

6. Synthetic data and practical analytics

Synthetic data is safe, fictional demo data. It should feel believable enough to support the story, but it must not contain real customer records, passwords, API keys, tokens, or payment details.

What makes synthetic data good?

Quality	Example
Connected	Every job points to a real synthetic client and account.
Consistent	Dates, IDs, services, and statuses follow the data contract.
Varied	Include completed, scheduled, cancelled, repeat-client, quiet, and busy cases.
Purposeful	Include enough activity to show weekly trends and at-risk accounts.
Safe	Use fictional names/contact details and no secrets.

Manual data workflow

Understand -> Inspect -> Clean -> Sort -> Filter -> Group -> Calculate -> Look for patterns -> Visualize -> Interpret.

Check	What to do
Row meaning	First define what one row represents. In a jobs table, one row should normally be one job.
Missing values	Ask whether the missing field matters for the analysis. A blank phone may not affect weekly job counts; a missing date does.
Duplicates	Remove genuine duplicate job IDs, but do not mistake a repeat client's separate jobs for duplicates.
Inconsistent categories	Standardize variants such as "complete," "done," and "Completed" to one allowed value.
Dates/outliers	Normalize date formats; sort price, dates, clients, services, and statuses to find suspicious values or useful patterns.

Choose the visualization from the question

Question	Best default presentation
How is activity changing over time?	Line chart.
Which service/category is largest?	Bar chart.
What is one important number?	KPI card, e.g., 18 jobs this week; +20% vs. last week.
Need exact individual records?	Table.
Are values spread across ranges?	Histogram.
Are two numeric variables related?	Scatter plot.

INTERPRET CAREFULLY

Observation: "Jobs fell from 17 to 6." **Interpretation:** "The account is definitely leaving."
Keep those separate; the MVP should flag a meaningful usage decrease, not claim certainty.

7. Chatbot choices and data safety

Approach	MVP fit	Recommendation
Rule-based / intent-based	Excellent: predictable help flows, buttons, and 10-20 common intents.	Safest default MVP. Use for focused support and account lookups.
Traditional ML	Useful later for classification or churn prediction when real labeled history exists.	Do not train a custom model for the initial chatbot.
LLM	Natural conversation and explanation; needs guardrails, costs, and safety design.	Optional enhancement. Use RAG for help documentation and backend tools for facts.

RAG is not fine-tuning

RAG (Retrieval-Augmented Generation) finds relevant approved documents at question time and gives them to an LLM. It is a strong way to answer “How do I reschedule?” from FieldFlow help content. **Fine-tuning** changes a model using training examples; it is not needed for this MVP and does not replace current, approved documentation.

Documentation question -> approved help content -> optional LLM -> answer + source
Account question -> authenticated backend lookup -> safe fact -> response
Write request -> Future: confirmation + strict server-side authorization

Security rules that matter

Rule	Why
Backend enforces account scope	Never rely on a prompt to stop John from seeing another contractor's data.
Least privilege	Give the chatbot only the specific account fields/tools it needs.
Data minimization	Send a safe count or plan name to an LLM, not every client record or sensitive detail.
Separate sources	Documentation belongs in a knowledge base; account facts come from authorized queries.
Read-only MVP	Answer questions and show safe information; avoid data-changing chatbot actions initially.
Safe fallback	If approved help content is not found, say so instead of inventing instructions.
Secrets stay server-side	Never expose database credentials, API keys, passwords, or tokens in the UI or mock data.

A SOUND MVP DECISION

A rule-based chatbot is fully defensible: it is easier to test, predictable, less likely to hallucinate, and can grow into LLM + RAG later.

8. Shared design, shared behavior, and folder structure

A short style guide plus a shared stylesheet/design system keeps four products from looking like four unrelated projects.

Shared design system	Product-specific styling
Colors, typography, buttons, inputs, cards, tables, navigation, badges, spacing, status colors, icons.	Calendar layout, job cards, analytics chart layout, chat bubbles/window, Ops account-table layout.

Design rule

Product CSS can extend the FieldFlow design system; it should not redefine it. Use shared variables such as `--color-primary` and `--radius-card`; avoid recreating global button styles inside a product stylesheet.

Shared behavior vs. product behavior

Shared services / utilities	Product-specific behavior
auth: login, logout, role checks; api: backend requests; notifications: success/error messages; utils: date formatting, validation, formatting, small reusable calculations.	Scheduling: job CRUD and conflict checks; Analytics: filters, calculations, charts; Chatbot: messages/lookups; Ops: filters, sorting, risk display.

A beginner-friendly structure

```
fieldflow/  
  src/  
    shared/  
      styles/fieldflow.css  
      components/  
      services/api.ts  
      services/auth.ts  
      services/notifications.ts  
      utils/dates.ts  
      utils/validation.ts  
      types/  
    scheduling/  
    ops-dashboard/  
    chatbot/  
    analytics/  
    shared-data/      # synthetic JSON / seed data  
    docs/              # standards, data model, API contract
```

Keep shared code genuinely shared

Put a function in **utils** only when multiple areas truly need the same small logic. Formatting a date is shared; “create a plumbing job” belongs in Scheduling. Services communicate with the outside world; utilities transform input into output.

BUILD ONLY WHAT YOU NEED

Start with a small structure. Extract shared services and utilities when repeated code appears; do not create empty abstractions just because they might be useful someday.

9. Team ownership, integration, and delivery

With a four-person team, do not create a separate person for every title. Give each person product ownership plus one shared responsibility.

Responsibility	What someone must actively own
Product / project lead	MVP scope, priorities, deadlines, dependencies, and keeping the assignment requirements visible.
Architecture / integration lead	Shared data model, backend decisions, interfaces, and verifying the four products work together.
Data / backend lead	Tables, synthetic data, API/data contracts, business definitions, and source-of-truth rules.
Frontend / UX / design lead	Shared components, accessibility, responsive behavior, John's workflow, style guide.
Security / auth lead	Roles, permissions, safe data access, secret handling, chatbot data boundaries.
QA / testing lead	Test plan and cross-product checks, including failures and edge cases.
Deployment / release owner	Ensure the integrated demo can run where the instructor can see it.
Documentation owner	Setup notes, architecture overview, data/API standards, and clear MVP decisions.
Demo / story owner	A single story that demonstrates why the products belong together.

Dependency plan



QA scenarios worth testing

Scenario	Expected result
John creates a job	Job persists; Schedule reflects it; Analytics count matches; Ops activity is updated.
Wrong account tries access	Backend blocks it; no other contractor's records are returned.
No jobs / duplicate times / backend failure	Clear empty, validation, and error states; the app does not silently lie.
Chatbot asks unknown question	Safe fallback; no invented instructions or leaked data.
At-risk rule	The same documented calculation produces the same flag every time.

DEMO STORY + FINAL CHECKPOINT

John logs in -> sees today's work -> adds a plumbing job -> schedule and analytics reflect it -> asks the chatbot for help -> later a meaningful activity decline appears in Internal Ops. Before building deeply, agree on the MVPs, roles, data contract, account boundaries, shared design tokens, and the 3-4 cross-product flows you will demonstrate.